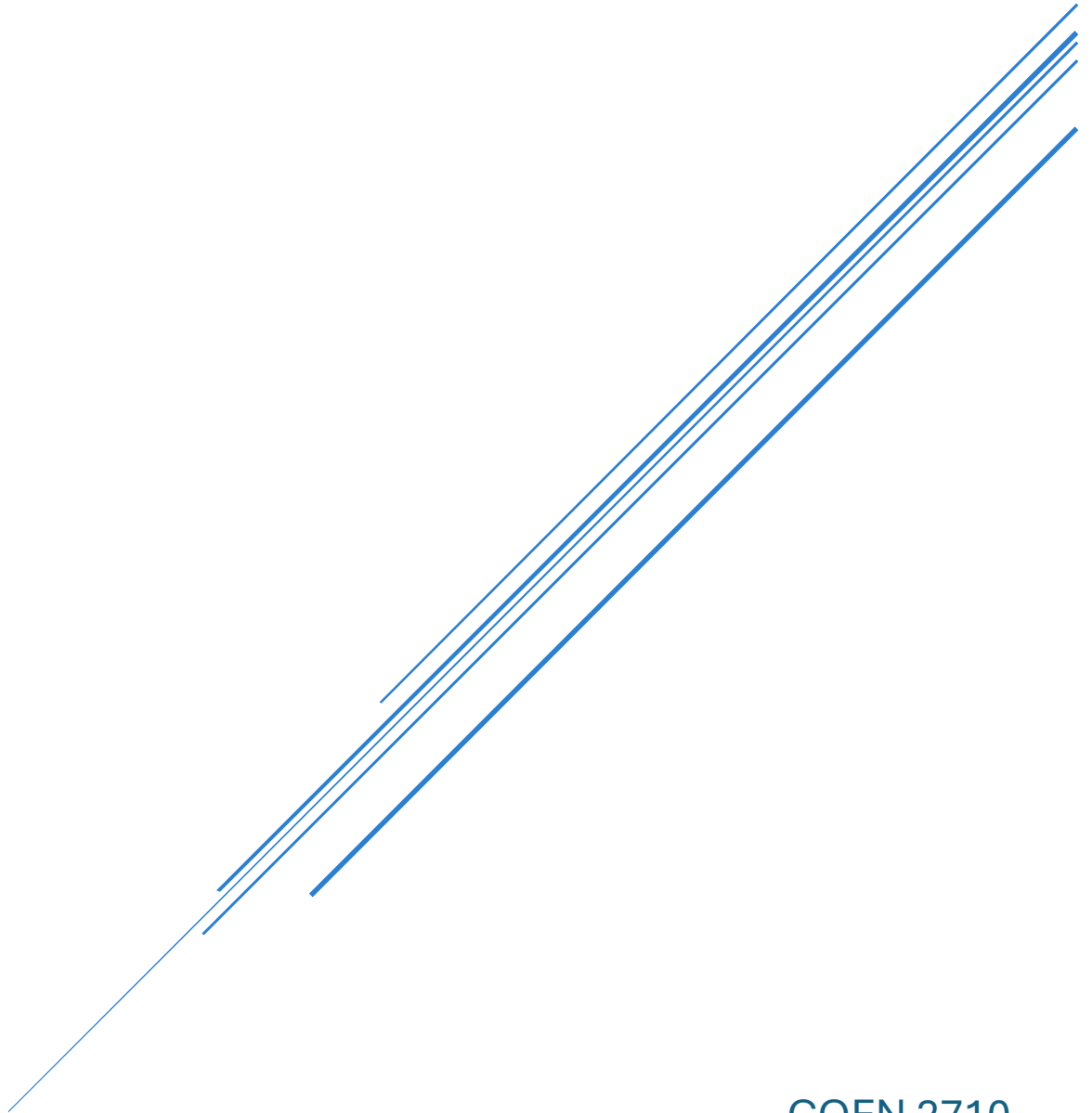


DESIGN PROJECT 2 PT.2

Microprocessors

22 April 2025



COEN 2710

Annabel Gabriel, Tyler Giczkowski, Aidan VandenElzen

Summary:

For design project 2, part 2, the team was tasked to create a simplified ALU (arithmetic-logic unit) component found within the data path for the Risk-V operations ADD, SUB, AND, OR, LW, SW, and BEQ. These components were created in Vivado in the language VHDL. The ALU has been successfully tested utilizing test benches and waveform charts to monitor proper input-output functions. For each component, using Vivado's report timing summary, the longest timing delay can be determined. For the ALU, the most critical path showed a timing delay of 14.792 ns.

Description:

ALU Component:

The ALU component is a part of the CPU (central processing unit). It completes all arithmetic and logic units within the Risk-V operations: ADD, SUB, AND, OR, LW, SW, BEQ, etc. The ALU utilizes an ALU_operation string, which is a 4-bit binary number that tells the ALU what operation to perform. For example, the operation 0011 corresponds to the operation ADD. This can be seen in Table 1.

Binary Inputs	ALU Operation
0000	AND
0001	OR
0010	ADD
0110	SUB
0111	SLT
1100	NOR

Table 1: Truth table for the ALU Operation with their respective inputs.

If that operation is detected, the add operation would be carried out on the two inputs a and b depicted in figure 1. The ALU includes inputs Input A, Input B, Carry In, and Control Flags. The outputs of the ALU include Result, overflow, zero and the Carry Out. The ALU was created by taking the 2-to-1 Mux given in the project 2 source code and creating a 1-bit ALU and a 1-bit ALU MSB. With those components a 32 bit ALU was created. Their inputs were all taken from one bit of the individual inputs and their outputs were concatenated together to create a final 32-bit result.

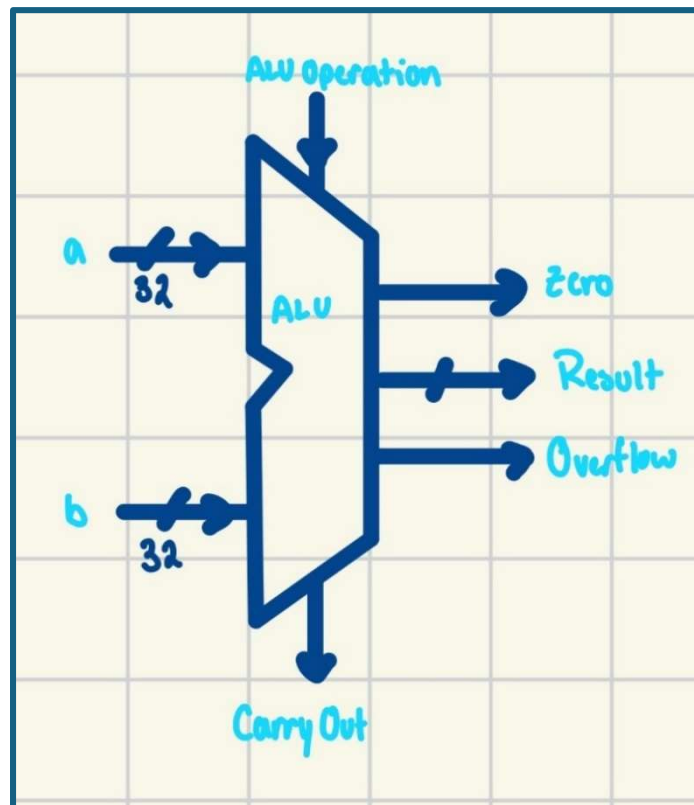


Figure 1: ALU Component Diagram

There are a few special exceptions that require additional inputs to receive the desired value in the Result output. In order to include subtraction, a 2-input mux is required to choose between ADD and SUB. This input is Binvert. To select an output function a 3-input mux is required. This input is Operation. In order to include NOR, both an Ainvert and a Binvert is required to select both NOT A and NOT B in their own 2-input muxes. The Operation input is also required once again. The SLT (Set on Less Than) operation requires Binvert once again, but it also requires a new input called Less that is set to logical 0. For all bits of the ALU operation except the MSB (most significant bit) this works. For the MSB, new outputs called Set and Overflow are required for sign detection and overflow. These are the new inputs for the final concatenated ALU that contains the MSB. A truth table for the ALU Operation is given in Table 1. Figure 2 depicts the synthesized diagram for the designed ALU provided by the Vivado tools, which demonstrates the architecture of the final implementation of the ALU is depicted in Figure 1.

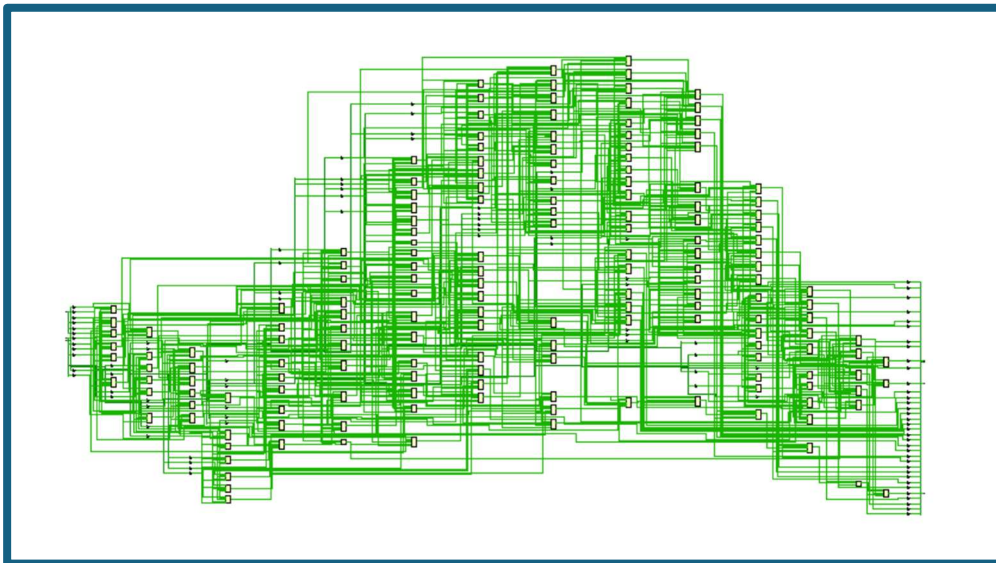


Figure 2: Schematic of the ALU Component

Testing and Verification with Results:

ALU Component:

The ALU component was tested using a testbench depicted in Figure 3. The inputs and outputs for the ALU component were noted from the truth table in Table 1.

At 0ns, the ALU Operation changed to 0010, which corresponds to the ADD operation. The inputs a and b are 5 and 3, respectively, and the result is 8. This operation is working correctly as $3+5=8$. The outputs carryout, result, and overflow are set to 0 which is correct based on this operation.

At 10ns, the ALU Operation changed to 0110, which corresponds to the SUB operation. The inputs a and b are “a” and “a”, respectively, and the result is 0. This operation is working correctly as $10-10=0$. The outputs are set to carryout=1, result=0, overflow=0, and zero is set to 1 which is correct based on this operation.

At 20ns, the ALU Operation changed to 0000, which corresponds to the AND operation. The inputs a and b are “...0f” and “...f0”, respectively, and the result is 0. This operation is working correctly as the inputs are not the same, so the result is 0. The outputs are set to carryout=0, result=0, overflow=0, and zero remains set to 1 which is correct based on this operation.

At 30ns, the ALU Operation changed to 0001, which corresponds to the OR operation. The inputs a and b are “...0f” and “...f0”, respectively, and the result is 0. This operation is working correctly as the inputs are “merged” due to the operation “...00ff”. The outputs are set to carryout=0, result=“...00ff,” overflow=0, and zero is falls to 0 which is correct based on this operation.

At 40ns, the ALU Operation changed to 0111, which corresponds to the SLT operation. The inputs a and b are 1 and 2, respectively, and the result is 1. This operation is working correctly as 1 is less than 2 and the result is 1. The outputs are set to carryout=0, result=1 overflow=0, and zero remains at 0 which is correct based on this operation.

At 50ns, the ALU Operation changed to 1100, which corresponds to the NOR operation. The inputs a and b are “ffffff” and 0, respectively, and the result is 0. This operation is working correctly as the inputs are not the same, so the result is 0. The outputs are set to carryout=0, result=0 overflow=0, and zero rises to 1 which is correct based on this operation.

At 60ns, the ALU Operation changed to 0110, which corresponds to the SUB operation. The inputs a and b are 0 and 0, respectively, and the result is 0. This operation is working correctly as the inputs are the same, so the result is 0. The outputs are set to carryout=1, result=0 overflow=0, and zero remains at 1 which is correct based on this operation.

Throughout the simulation, the ALU Component correctly generated signals according to the truth table, ensuring the processor's correct behavior based on instruction opcodes. The observed results validate that the ALU Component is functioning as expected.

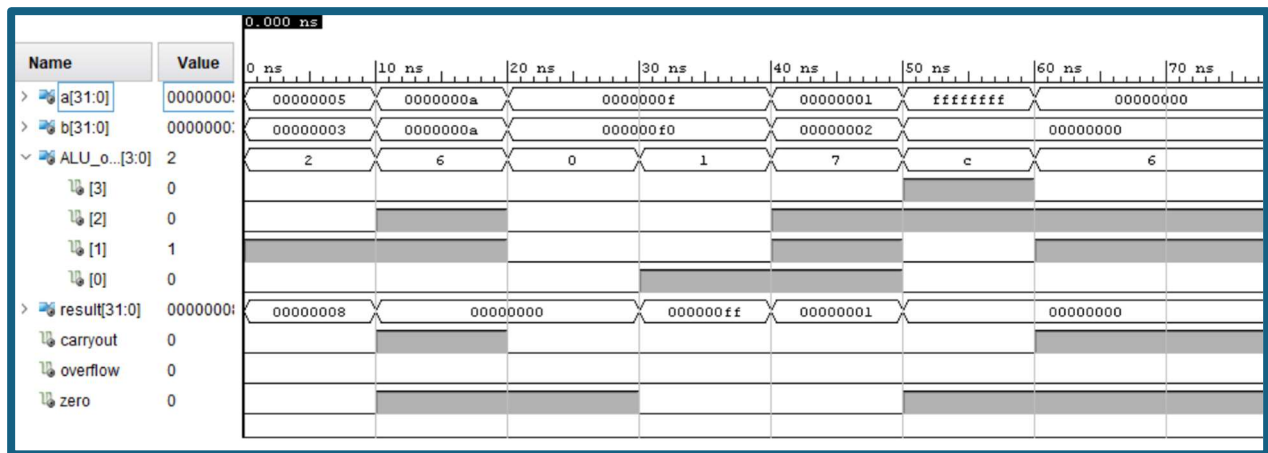


Figure 3: Test bench simulation for the ALU Component.

Performance Analysis:

ALU Component:

The timing analysis for the ALU component revealed the longest timing delay for the circuit was from the input ALUOp to the output Zero. The total delay of this path was 14.792ns. Figure 4 demonstrates this by showing only the most-critical path that this delay time references, starting at ALUOp, and ending at Zero.

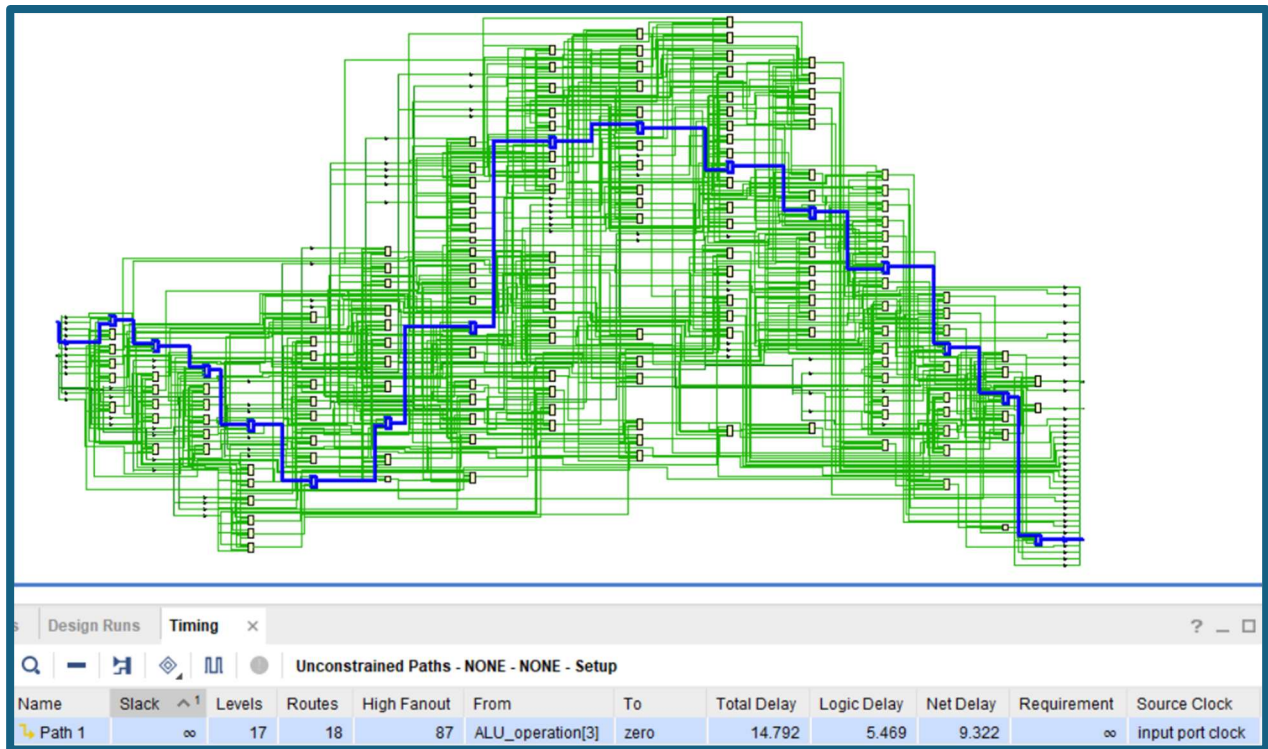


Figure 4: Schematic diagram depicting the longest path by the blue line.

Conclusions:

The purpose of this project was to design and implement the ALU component of the RISC-V processor. To do this, entity declarations and descriptions were made in VHDL to describe the ALU component. The team used *Xilinx's Vivado WebPack* to design and simulate the components using VHDL description language [1]. These descriptions were made from analyzing diagrams and data paths for implementation given in the lecture *Arithmetic for Computers – Building the ALU* (Ababei, C.).

Each member of the group worked to translate the data path diagrams into an entity description based on how the ALU component was implemented. This format is referenced and executed in the textbook *DIGITAL DESIGN USING DIGILENT FPGA BOARDS: Vhdl* (Haskell, R. 2017) [3]. This taught to not only to examine the truth data path execution but to understand and develop digital logic to test the functions in simulations. With this came new understanding of running simulations on VHDL software and performance analysis.

Challenges came when attempting to meet and work on the project together as each member has busy schedules. However, most of the work completed during this project was done independently and then discussed and observed when the team had the opportunity to be together and work on the project. Other challenges included hidden syntax errors or incorrect analysis of the Boolean functions that through off testing and measurements. Team members struggled with understanding the logic of certain parts of the component. However, through working together, a consensus was able to be reached in order to determine the desired output. Paired programming was very useful to find either the syntax error, incorrect input, or logic.

References:

- [1] *Installer Information*. (n.d.). AMD. <https://www.xilinx.com/support/installer/installer-info-general.html>
- [2] Ababei, C. *Arithmetic for Computers – Building the ALU (Ch.3 + Appendices A,C)*
- [3] Haskell, R. (2017). *DIGITAL DESIGN USING DIGILENT FPGA BOARDS: Vhdl*.